

International Computer Science Competition

Solution for problems Qualification Round



PROBLEM A SOLUTION:

Identify and label the following components based on the figure above:

- 1) $w^{(1)}$: the weight from the 2nd input to the 1st hidden neuron in layer 1.
- 2) Σ : the **weighted sum (pre-activation)** into a neuron.
- 3) f : the neuron's **activation function** (nonlinearity), such as sigmoid.
- 4) : an **input node / feature**.
- 5) : a **bias node** providing b to the neuron.
- 6) : the **output neuron**.
- 7) Box A: the **hidden layer (layer 1)**.
- 8) Box B: the **output layer (layer 2)**.
- 9) $\hat{y}$: the model's **prediction or output**.

PROBLEM B SOLUTION:

```

26  # --- WRITE YOUR CODE HERE ---
27  # Validate inputs
28  if flour <= 0 or sugar <= 0:
29      | raise ValueError("Flour and sugar must be positive integers.")
30
31  # Recipe requirements
32  flour_needed = 100
33  sugar_needed = 50
34
35  # Maximum cakes that can be baked
36  cakes = min(flour // flour_needed, sugar // sugar_needed)
37
38  # Leftover ingredients
39  remaining_flour = flour - (cakes * flour_needed)
40  remaining_sugar = sugar - (cakes * sugar_needed)
41
42  # Return result
43  return [cakes, remaining_flour, remaining_sugar]
44

```

PROBLEMS 8 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS + v

```

Input Error or Validation Failed: invalid literal for int() with base 10: ''
100
(.venv) PS C:\Users\dell\Desktop> python .\problem_b.py
100
50
1 0 0

```

EXPLANATION:

The number of cakes that can be baked without losing any ingredients is what is done in The Cake Calculator. It is understood that one cake needs 100 units of flour and 50 units of sugar. It needs to be understood that the the number of cakes baked is determined by the scarcest resource.

The process is simple. The first thing to do is to ascertain that the amounts of flour and sugar is a positive number because without those two baking cannot be done. After that, we check how many possible cakes can be baked based on the flour available and how many can be baked based on the sugar available and we take the limit based on the lower number.

The available ingredients will be reduced by the number of baked cakes. It is easy to see what the leftover ingredients are by comparing the stock ingredients and the ingredients that have been used.

Any solution to the problem is inherently interesting in itself, but what I find particularly engaging is how closely the solution aligns with the logic of actual baking. It is not as if one ingredient is looked at in isolation. If you run out of sugar first, then sugar controls how many cakes you can bake, regardless of the fact that you have plenty of flour. The converse is also true. Such a way of thinking, although not particularly sophisticated, is what makes the algorithm extremely easy to comprehend.

To clarify further, consider the following scenarios:

You have 100 flour and 50 sugar. You can bake 1 and only 1 cake, and this is with no cakes left over.

PROBLEM C SOLUTION:

ANSWER 1:

Standard text encodings such as ASCII consider each character as taking the same number of bits, irrespective of the number of occurrences of each character. This is inefficient as common characters take up more bits than needed.

The average number of bits per character can be reduced by using variable-length codes in which shorter codes are assigned to common characters. This allows more symbols to be sent in the same amount of data by more efficiently utilizing the upper limit on the number of bits per character.

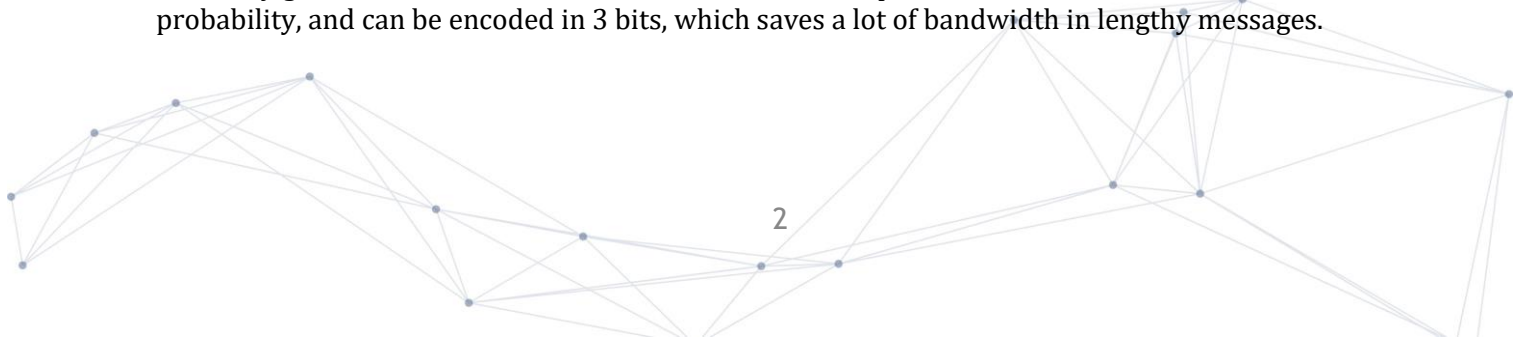
Suppose we encode characters A (70%) and B (30%) for example:

In fixed-length encoding, both are assigned 1 bit (e.g., A = 0 and B = 1), hence the average length for both is 1 bit per character.

Variable-length encoding assigns A = "0" (1 bit) and B = "10" (2 bits).
The average length becomes:

$$(0.7 \times 1) + (0.3 \times 2) = 0.7 + 0.6 = 1.3 \text{ bits per character}$$

In the example within which this two character example is applied to the 12 character problem, the efficiency gains becomes self-evident. Character A is a frequent character as it has a 0.20 probability, and can be encoded in 3 bits, which saves a lot of bandwidth in lengthy messages.



ANSWER 2: We compute entropy using the formula:

$$H = - \sum_{i=1}^n p_i \times \log_2(p_i)$$

Each symbol's contribution (rounded) is:

- A (0.20): 0.46439
- B (0.15): 0.41054
- C (0.12): 0.36707
- D (0.10): 0.33219
- E (0.08): 0.29151
- F (0.06): 0.24353
- G (0.05): 0.21610
- H (0.05): 0.21610
- I (0.04): 0.18575
- J (0.03): 0.15177
- K (0.02): 0.11288
- L (0.10): 0.33219

Adding these:

$H \approx 0.46439 + 0.41054 + 0.36707 + 0.33219 + 0.29151 + 0.24353 + 0.21610 + 0.21610 + 0.18575 + 0.15177 + 0.11288 + 0.33219$

$H \approx 3.324$ bits per character

Interpretation (what this represents for optimal encoding):

- The entropy $H \approx 3.324$ bits/character is the theoretical lower bound on the average number of bits needed to encode this symbol set losslessly.
- No lossless prefix code (instantaneously decodable code) can have an average length smaller than this value. Practical prefix codes (e.g., Huffman or Fano) aim to approach this bound; the smaller the gap between a code's average length and H , the more efficient the code.

ANSWER 3:

Symbol	p_i	Fano Length	$p_i \times l_i$
A	0.20	3	0.600
B	0.15	3	0.450
C	0.12	3	0.360
D	0.10	4	0.400
E	0.08	4	0.320
F	0.06	4	0.240
G	0.05	3	0.150
H	0.05	4	0.200
I	0.04	4	0.160
J	0.03	4	0.120

Symbol	pi	Fano Length	pi×li
K	0.02	4	0.080
L	0.10	4	0.400

Average L bar = 3.48 bits/char

- Entropy: $H \approx 3.324$ bits/char
- Redundancy: $3.48 - 3.324 = 0.156$ bits/char
- Efficiency: $3.324 \div 3.48 \approx 95.5\%$

Conclusion: Fano code is near-optimal with 3.48 bits/char ($\approx 95.5\%$ efficient). Huffman code gives 3.37 bits/char ($\approx 98.6\%$ efficient), even closer to entropy.

PROBLEM D SOLUTION:

```

17     size = 10
18     grid = [[' ' for _ in range(size)] for _ in range(size)]
19
20     directions = [(0, 1), (1, 0), (1, 1), (-1, 1)] # right, down, diagonal-down, diagonal-up
21
22     def can_place(word, x, y, dx, dy):
23         for i, ch in enumerate(word):
24             nx, ny = x + i * dx, y + i * dy
25             if nx < 0 or ny < 0 or nx >= size or ny >= size:
26                 return False
27             if grid[nx][ny] not in (' ', ch):
28                 return False
29         return True
30
31     def place_word(word):
32         for _ in range(200):
33             x, y = random.randint(0, size - 1), random.randint(0, size - 1)
34             dx, dy = random.choice(directions)
35             if can_place(word, x, y, dx, dy):
36                 for i, ch in enumerate(word):
37                     nx, ny = x + i * dx, y + i * dy
38                     grid[nx][ny] = ch
39                 return True
40         return False
41
42     for word in words:
43         word = word.lower().strip()
44         if word:
45             if not place_word(word):
46                 place_word(word[::-1])
47
48     letters = 'abcdefghijklmnopqrstuvwxyz'
49     for i in range(size):
50         for j in range(size):
51             if grid[i][j] == ' ':
52                 grid[i][j] = random.choice(letters)
53
54     return grid
55
56

```

PROBLEMS 8 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

IndentationError: unindent does not match any outer indentation level
(.venv) PS C:\Users\dell\Desktop> python .\problem_d.py
● learning, code, machine
hyimachine
jzggrgzxtb
zqcaoe knht

```

EXPLANATION:

A 10×10 word search puzzle is generated using the following approach:

Grid Initialization: Create a 10×10 empty grid, which is a list of lists.

Direction Setup: Define four directions: horizontal (right), vertical (down), diagonal-down, and diagonal-up.

Placement Check (can_place): Ensure that every letter of a word will fit into the grid without overlapping a conflicting letter.

Word Placement (place_word): Place each word at a randomly chosen valid starting position and direction. If a word cannot be placed after a maximum of n attempts, the reverse order of the word is tried.

Fill Remaining Cells: After all the words are placed, the empty cells are filled with random letters to complete and improve the challenge of the puzzle.

Return the Grid: The function returns a 2D grid with words and random letters used as fillers.

What Makes This Solution Outstanding

Random Placement: Each puzzle created is unique to every session, adding to creativity.

Multi-directional: Words are placed and hidden in horizontal, vertical, and diagonal directions, increasing puzzle difficulty.

Error protection: The solution accounts for and avoids erroneously placing puzzle pieces outside the grid or overlapping them.

Completeness of the puzzle: All empty spaces are filled, with random letters in every cell.

PROBLEM E SOLUTION:

Step 1: Representing NOT Using NAND

From Idempotent Law: $P \equiv P \wedge P$

Apply negation: $\neg P \equiv \neg(P \wedge P)$

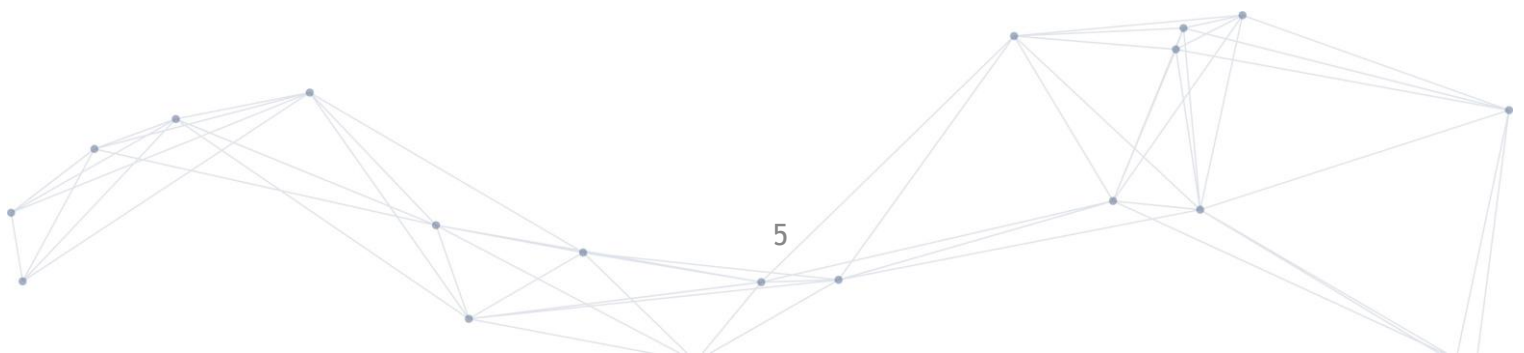
By NAND definition: $\neg P \equiv P \uparrow P = \text{NAND}(P, P)$

Step 2: Representing AND Using NAND: $P \wedge Q = \neg(P \uparrow Q) = (P \uparrow Q) \uparrow (P \uparrow Q)$

Step 3: Representing OR Using NAND

1) Using De Morgan's Law: $P \vee Q = \neg(\neg P \wedge \neg Q)$

2) Using NAND: $P \vee Q = (P \uparrow P) \uparrow (Q \uparrow Q)$



Truth Table (for NOT using NAND):

Truth Table (for NOT using NAND)		
P	$P \wedge P$	$\neg(P \wedge P) = P \text{ NAND } P$
0	0	1
1	1	0

Circuit Diagram (Representation)

- **NOT Gate** → One NAND gate with both inputs connected to P.
- **AND Gate** → Two NAND gates: first produces $(P \uparrow Q)$, second inverts it.
- **OR Gate** → Three NAND gates: two for $\neg P$ and $\neg Q$, one to combine.

Conclusion:

Since we can implement **NOT**, **AND**, and **OR** using only NAND gates, **NAND is functionally complete** meaning any Boolean function can be created using NAND alone.





